

WeatherWise AI

Backend Architecture & System Specifications Document

TEAM ID
SWTID-2026-8956

TEAM SIZE
4 Members

DATE
29/09/2026

VERSION
v1.0.0

Engineering Team Roster

Leader: SANMATHI KUMAR

ARIFA ZOHRA

AINUL LIYANA P M

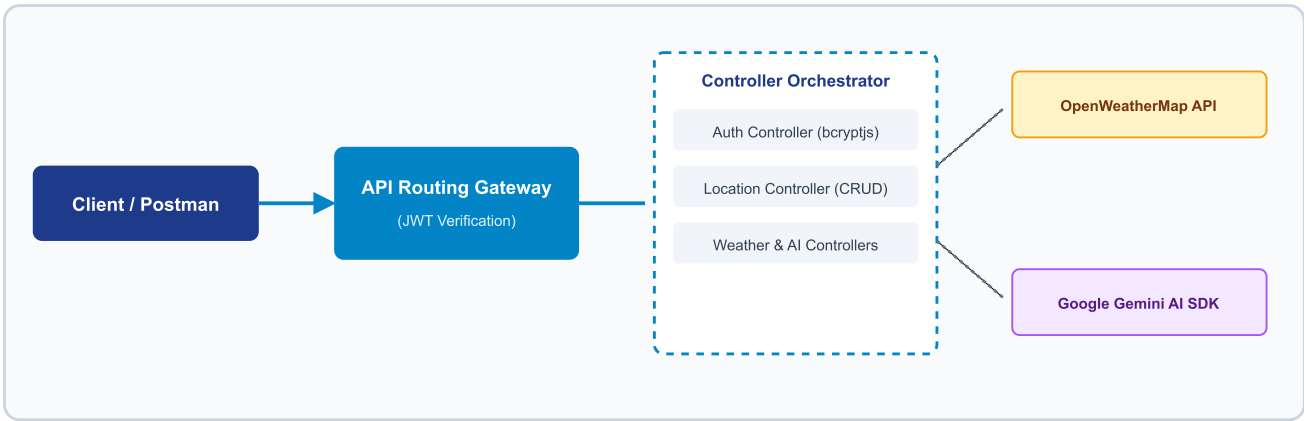
VAISHALI S

1. Executive Overview

WeatherWise AI is a RESTful backend service engineered to deliver real-time environmental metrics coupled with natural language summaries and activity recommendations derived from live atmospheric data. By combining OpenWeatherMap data with the Google Gemini AI SDK, the system solves key traveler pain points: multi-location tracking, automated data interpretation, and context-aware lifestyle recommendations.

2. Technical Architecture & Component Flow

The service implements a classic **Model-View-Controller (MVC)** architectural pattern, separating routing, business orchestration, and database logic.



3. Stack & Hardware Specifications

Software Stack

- **Runtime:** Node.js (v16+) & Express.js
- **Database:** MongoDB with Mongoose ODM
- **AI SDK:** `@google/generative-ai`
- **Security:** JWT & `bcryptjs`
- **Testing:** Postman / Thunder Client

Hardware Requirements

- **OS:** Windows 10/11, macOS, Linux
- **CPU:** Intel i5 (8th Gen+) / Ryzen 5
- **RAM:** 8 GB Minimum (16 GB Rec.)
- **Storage:** 1 GB Workspace

4. Database Schemas (Mongoose ODM)

1. User Schema (MongoDB Collection: `users`)

Field	Type	Constraints & Validation
<code>_id</code>	ObjectId	Primary Key (Auto-generated)
<code>name</code>	String	Required
<code>email</code>	String	Required, Unique, Lowercase, Regex Validated
<code>password</code>	String	Required, Hashed via <code>bcryptjs</code> , Min length 6
<code>timestamps</code>	Date	Managed automatically (<code>createdAt</code> , <code>updatedAt</code>)

2. Location Schema (MongoDB Collection: `locations`)

Field	Type	Constraints & Validation
<code>_id</code>	ObjectId	Primary Key (Auto-generated)
<code>city</code>	String	Required
<code>country</code>	String	Required
<code>user</code>	ObjectId	Foreign Key referencing <code>User</code> collection (Required)

Index Constraint: Compound unique index `{ user: 1, city: 1, country: 1 }` to prevent duplicate location entries per user.

5. Roles & Core End-to-End Workflow

Role	Permissions & Core Capabilities
ADMIN	Manages global accounts, oversees database integrity, monitors API health/logs.
REGISTERED USER	Manages favorite cities (CRUD), requests weather metrics, queries Gemini AI for advice.
PUBLIC USER	Interfaces with open endpoints to view raw weather data without authentication.